

บทที่ 2

ทฤษฎีพื้นฐานที่ใช้ในโครงการ

ก่อนที่จะทำการออกแบบหรือพัฒนาระบบใดๆ ขึ้นมานั้น การศึกษาหาข้อมูลของทฤษฎีที่เกี่ยวข้องนั้นเป็นสิ่งที่จะต้องกระทำอย่างหลีกเลี่ยงไม่ได้ เพื่อให้ระบบที่ต้องการออกแบบหรือพัฒนานั้นประสบผลสำเร็จสูงสุดและเกิดความผิดพลาดน้อยที่สุด

ในบทนี้จะกล่าวถึงทฤษฎีต่างๆ ที่เกี่ยวข้องกับการทำระบบเครือข่ายสำหรับโปรแกรมบิตทอเรนซ์ ได้แก่ โปรโตคอลบิตทอเรนซ์ องค์ประกอบและการทำงานของบิตทอเรนซ์ พร็อกซี เซิร์ฟเวอร์ โปรแกรม Squid โปรแกรม SquidGuard และเว็บเซิร์ฟเวอร์ ซึ่งทฤษฎีที่เกี่ยวข้องนี้จะนำไปใช้ในการออกแบบระบบและพัฒนาระบบในบทต่อไป

2.1 บิตทอเรนซ์

2.1.1 บิตทอเรนซ์คืออะไร

BitTorrent เป็นชื่อของโปรโตคอลและอุปกรณ์การแจกจ่ายข้อมูลแบบ Peer-to-peer ที่ถูกพัฒนาโดยผู้พัฒนา Bram Cohen โดยเริ่มเป็นที่รู้จักครั้งแรกในงาน CodeCon ปี ค.ศ. 2002 ในครั้งแรกการทำงานเขียนโดยภาษาไพทอนและจัดสิทธิบัตรในแบบโอเพ่นซอร์สภายใต้เวอร์ชัน 4.0 คำว่า BitTorrent สามารถใช้อ้างถึงตัวโปรโตคอลในการแจกจ่ายไฟล์ ตัวโปรแกรมประยุกต์ส่วนลูกข่าย และไฟล์นามสกุล .torrent ได้

โปรโตคอล BitTorrent จะทำการแบ่งไฟล์ออกเป็นหลายๆ ส่วนย่อยๆ โดยประมาณคือหนึ่งในสี่ของเมกะไบต์ แต่ละส่วนย่อยจะถูกแจกจ่ายไปยังเครื่อง peer โดยสุ่มลำดับ ซึ่งจะประกอบกลับเป็นไฟล์หนึ่งในเครื่องที่ขอไฟล์ไป แต่ละ peer จะใช้ประโยชน์จากการเชื่อมต่อที่ดีที่สุดไปยังส่วนที่ยังขาดไป ในขณะที่ส่งส่วนที่มีอยู่ให้เครื่องอื่นๆ ได้รับ รูปแบบการทำงานอย่างนี้มีประโยชน์อย่างมากในการแลกเปลี่ยนไฟล์ขนาดใหญ่ อย่างเช่นวิดีโอหรือโค้ดต้นแบบของโปรแกรม ในการดาวน์โหลดโดยทั่วไปแล้ว ไฟล์ที่มีความต้องการสูงมักก่อให้เกิดสถานะคอขวด ซึ่งผู้ที่ต้องการไฟล์จะใช้แบนวิดท์ของเครื่องให้บริการจนหมด แต่ใน BitTorrent นั้นไฟล์ที่เป็นที่ต้องการอย่างมากจะถูกช่วยในการส่งไฟล์เมื่อแบนวิดท์และจำนวนเครื่องปล่อยไฟล์ (seed) ที่สมบูรณ์มีมากขึ้น

การทำงานของ BitTorrent เครื่องลูกข่ายจำเป็นต้องมีสิ่งหลักๆ พื้นฐานก็คือโปรแกรมประยุกต์ที่รองรับโปรโตคอล BitTorrent ไฟล์นามสกุล .torrent และการเชื่อมต่อกับอินเทอร์เน็ต

2.1.2 รายละเอียดของไฟล์ .torrent

ไฟล์นามสกุล .torrent จะมีข้อมูลที่เกี่ยวข้องกับไฟล์จริงๆที่จะทำการดาวน์โหลด เรียกข้อมูลนี้ว่า Metainfo ซึ่งถูกจัดรูปแบบในลักษณะ Bencode (อ่านว่า บี-เอนโค้ด) โดยมีรายละเอียดของรูปแบบดังนี้

- ข้อมูลที่เป็น String จะอยู่ในรูป <ความยาวของstringในเลขฐานสิบ>:<ข้อมูลstring> เช่น 4:spam
- ข้อมูลที่เป็นจำนวนเต็มจะอยู่ในรูป i<จำนวนเต็มในAsciiฐานสิบ>e โดยสามารถใส่เครื่องหมายบอกจำนวนลบไว้ก่อนตัวเลขได้ จำนวนเต็มศูนย์ให้ใส่เพียงเลขศูนย์ เช่น i7e i-3e i0e
- ชุดข้อมูล (list) จะอยู่ในรูป l<ข้อมูลในรูปแบบbencode>e ซึ่งในชุดข้อมูลสามารถประกอบไปด้วยข้อมูลในรูปแบบ bencode ที่เป็น string จำนวนเต็ม พจนานุกรม และ/หรือ ชุดข้อมูลอื่นๆก็ได้
- พจนานุกรม จะอยู่ในรูป d<bencoded string><bencoded element>e โดยตัวตั้งต้นจะต้องเป็น string ส่วนค่านั้นจะเป็นอะไรก็ได้ที่อยู่ในรูปแบบ bencode และหากมีมากกว่าหนึ่งตัวตั้งต้นจะต้องทำการจัดลำดับตัวตั้งต้นตามลำดับ alphabet ของ string เช่น d4:bone7:calcium3:cow3:mooe ซึ่งก็คือ bone=calcium และ cow=moo

โครงสร้างของ Metainfo ทุกอย่างจะอยู่ในรูปของ bencode โดย metainfo ในไฟล์นามสกุล .torrent เป็น bencoded dictionary ที่มีข้อมูลกล่าวไว้ด้านล่าง ทุกตัวอักษร string อยู่ในรูป UTF-8 โดยส่วนใดไม่ได้ระบุว่าเป็นตัวเลือกคือข้อมูลที่จำเป็น

- info: เป็น dictionary ที่บรรยายลักษณะของไฟล์หนึ่งๆหรือหลายๆไฟล์ใน torrent ซึ่งสามารถมีได้สองลักษณะ คือแบบไฟล์เดี่ยวที่ไม่ต้องมีโครงสร้าง Directory กับแบบไฟล์ torrent ที่บอกถึงหลายไฟล์

สำหรับไฟล์เดี่ยว รูปแบบพจนานุกรม info จะมีโครงสร้างดังรูปที่ 2.1 นี้

Info	Length	MD5 Checksum	Name	Piece	Pieces
Announce	Announce List	Create Date	Comment	Created by	

รูปที่ 2.1 รูปแบบพจนานุกรม info สำหรับไฟล์เดี่ยว

- length: ความยาวของไฟล์ (จำนวนเต็ม)
- md5sum: (ตัวเลือก) string ความยาว 32 ตัวอักษรในรูปเลขฐาน 16 ที่ตรงกับ MD5 Sum ของไฟล์ ข้อมูลนี้แทบไม่ถูกใช้โดยโปรแกรม BitTorrent เลย แต่ถูกรวมไว้ในบางโปรแกรมเพื่อเพิ่มความยืดหยุ่นในการใช้งาน
- name: ชื่อของไฟล์ (string)
- piece length: ความยาวของแต่ละชิ้นส่วนของไฟล์ในหน่วยไบต์ ตัวเลขจะเป็นกำลังของ 2 ขึ้นไป โดยปกติจะตั้งไว้ให้น้อยกว่าหรือเท่ากับ 512 KB (จำนวนเต็ม)
- pieces: string ที่มีค่า SHA-1 hash ของแต่ละชิ้น โดยค่าเป็น 20 bytes ต่อหนึ่งชิ้น ทั้งหมดต่อกันเป็น string เดียวที่มีความยาวเป็นผลคูณของ 20

สำหรับไฟล์หลายไฟล์ รูปแบบพจนานุกรม info จะมีโครงสร้างดังรูปที่ 2.2 นี้

Info				
Files		Length	MD5 Checksum	
Path	Name	Piece Length	Pieces	
Announce	Announce List	Create Date	Comment	Created by

รูปที่ 2.2 รูปแบบพจนานุกรม info สำหรับไฟล์หลายไฟล์

- files: ชุดข้อมูลของพจนานุกรม แต่ละอันสำหรับแต่ละไฟล์ แต่ละพจนานุกรมจะมีชุดข้อมูลดังนี้
 - length: ความยาวของไฟล์ (จำนวนเต็ม)
 - md5sum: (ตัวเลือก) string ความยาว 32 ตัวอักษรในรูปเลขฐาน 16 ที่ตรงกับ MD5 Sum ของไฟล์ ข้อมูลนี้แทบไม่ถูกใช้โดยโปรแกรม BitTorrent เลย แต่ถูกรวมไว้ในบางโปรแกรมเพื่อเพิ่มความยืดหยุ่นในการใช้งาน
 - path: ชุดข้อมูลที่บันทึกหนึ่ง string หรือมากกว่าเพื่อบอก path และชื่อไฟล์ แต่ละค่าจะบอกถึงชื่อของ directory หรือ (ค่าสุดท้ายในชุดข้อมูล) ชื่อไฟล์
 - name: ชื่อของ directory บนสุดของโครงสร้าง—Directory ที่มีไฟล์ทุกไฟล์ตามที่กล่าวไว้ในชุดข้อมูล files

- piece length: ความยาวของแต่ละชิ้นส่วนของไฟล์ในหน่วยไบต์ ตัวเลขจะเป็นกำลังของ 2 ขึ้นไป โดยปกติจะตั้งไว้ให้น้อยกว่าหรือเท่ากับ 512 KB (จำนวนเต็ม)
- pieces: string ที่มีค่า SHA-1 hash ของแต่ละชิ้น โดยค่าเป็น 20 bytes ต่อหนึ่งชิ้น ทั้งหมดต่อกันเป็น string เดียวที่มีความยาวเป็นผลคูณของ 20

หลังจาก info จะเป็นรูปแบบที่ทั้งไฟล์เดี่ยวและหลายไฟล์ใช้เหมือนกัน

- announce: URL ของ tracker (string)
- announce-list: (ตัวเลือก) ส่วนขยายของข้อมูลปกติเพื่อบอก tracker สำรองในกรณีที่ tracker หลักล่มหรือติดต่อไม่ได้
- create date: (ตัวเลือก) เวลาที่สร้างไฟล์นามสกุล .torrent นี้ขึ้น ใช้รูปแบบ Unix epoch โดยนับเวลาเป็นวินาทีจาก 1 Jan 1970 ณ เวลา 00:00:00 UTC (จำนวนเต็ม)
- comment: (ตัวเลือก) ข้อความของผู้สร้าง (string)
- created by: (ตัวเลือก) ชื่อและรุ่นของโปรแกรมที่ใช้สร้างไฟล์ .torrent นี้ (string)

2.1.3 การทำงานของแทรคเกอร์

ในการเชื่อมต่อระหว่างแต่ละเครื่อง Peer นั้น จะมี BitTorrent Tracker เป็นตัวช่วยในการแลกเปลี่ยนข้อมูลสถานะของแต่ละ peer เพื่อให้ทราบว่า มีเครื่องใดบ้างที่ต้องการหรือมีไฟล์ดังกล่าวที่ตรงกับไฟล์นามสกุล .torrent บอกไว้ แทรคเกอร์มีหลายแบบ ที่พัฒนาแรกสุดก็คือ HTTP/HTTPS protocol Tracker พัฒนาในภาษาไพทอน ซึ่งตอบสนองต่อคำสั่ง HTTP GET คำสั่งที่ส่งมาจะมี ค่า metrics จากเครื่องลูกข่ายที่ช่วยแทรคเกอร์ในด้านสถิติของ torrent นั้นๆ การตอบสนองจะมี peer list ที่ช่วยให้เครื่องลูกข่ายทราบถึงเครื่องอื่นๆที่เข้าร่วมงาน torrent นี้ ปกติแล้วคำสั่งจะใช้ URL จาก announce ใน metainfo ตามด้วย Parameter โดยใช้วิธีปกติของ CGI (ใช้ ? หลัง announce URL ตามด้วยชุด parameter=value ซึ่งขึ้นจากกันด้วย &) ซึ่งค่าทั้งหมดนั้น ถ้าไม่ได้อยู่ในรูป 0-9 a-z A-Z และ \$-_.+!*() จะต้องอยู่ในรูป %nn โดยที่ nn เป็นค่าเลขฐานสิบหกที่ตรงกับไบต์ตัวอักษรนั้นๆ

Parameter ที่ใช้ใน คำสั่ง GET จากเครื่องลูกข่ายไปยัง Tracker จะมีข้อมูลดังรูปที่ 2.3 นี้

Info_Hash	Peer_ID	Port	Uploaded	Downloaded
-----------	---------	------	----------	------------

Left	Compact	Event	IP	Numwant	Key	TrackerID
------	---------	-------	----	---------	-----	-----------

รูปที่ 2.3 พารามิเตอร์ที่ใช้ส่งจากเครื่อง Client ไปยัง Tracker

- info_hash: 20 ไบต์ SHA-1 Hash ของค่าของ info ตามในพจนานุกรมใน metainfo แต่ string ที่ใช้นี้จะแปลงเป็น urlencoded string
- peer_id: 20 ไบต์ string ใช้เป็นหมายเลขเฉพาะของเครื่องลูกข่าย สร้างโดยโปรแกรมลูกข่ายตอนที่เริ่มโปรแกรม สามารถเป็นค่าใดๆก็ได้
- port: หมายเลขพอร์ตที่เครื่องลูกข่ายรับฟังอยู่พอร์ตที่จัดไว้สำหรับ BitTorrent โดยเฉพาะโดยปกติคือ 6881 ถึง 6889 เครื่องลูกข่ายอาจต้องยกเลิกหากไม่สามารถใช้พอร์ตในช่วงนี้ได้
- uploaded: จำนวนข้อมูลที่ได้ส่งออกไปให้เครื่องอื่นในรูปแบบ ASCII ฐานสิบ
- downloaded: จำนวนข้อมูลที่ได้โหลดเข้ามาในรูปแบบ ASCII ฐานสิบ
- left: จำนวนข้อมูลคงเหลือที่เครื่องลูกข่ายจำเป็นต้องโหลดในรูปแบบ ASCII ฐานสิบ
- compact: บ่งบอกว่าเครื่องลูกข่ายรับข้อมูลตอบรับที่กระชับ Peers list จะถูกแทนด้วย Peers string ข้อมูลที่ใช้คือ 6 ไบต์ต่อหนึ่ง peer โดย 4 ไบต์แรกเป็น host และอีกสองไบต์เป็นพอร์ต
- event: ถ้ามี จะต้องเป็นหนึ่งใน started, stopped หรือ completed ถ้าไม่ระบุ จะดำเนินการกระทำตามปกติ
 - started: คำขอครั้งแรกที่ติดต่อกับแทรคเกอร์จะต้องมี event ที่มีค่า started เสมอ
 - stopped: จะต้องส่งไปให้แทรคเกอร์ หากโปรแกรมลูกข่ายปิดลงด้วยดี
 - completed: จะต้องส่งไปให้แทรคเกอร์ เมื่อการดาวน์โหลดเสร็จสิ้นโดยสมบูรณ์ แต่ไม่ต้องส่งหากในตอนแรกที่เริ่มโปรแกรมลูกข่ายนั้นงานเสร็จสิ้นอยู่แล้ว
- ip: ตัวเลือก เป็นค่าไอพีแอดเดรสจริงของเครื่องลูกข่ายในรูปแบบ dotted quad หรือ hexed IPv6 ปกติไม่จำเป็นเพราะสามารถดูไอพีที่ request มาได้ เว้นแต่เป็นการติดต่อภายใต้พร็อกซี่ บางแทรคเกอร์ยังถือเป็นการบอกลักษณะการติดต่อแบบ IPv6 ด้วย
- numwant: ตัวเลือก ระบุจำนวนของ peer ที่เครื่องลูกข่ายต้องการรับจากแทรคเกอร์ ซึ่งสามารถเป็นค่าศูนย์ได้ หากไม่ระบุ ปกติจะรับ 50 peers

- key: ตัวเลือก ข้อมูลระบุตัวนี้จะไม่ใช้ร่วมกับผู้อื่น ถูกกำหนดมาเพื่อระบุตัวตนของเครื่องลูกข่ายหากมีการเปลี่ยนแปลงไอพีแอดเดรส
- trackerid: ตัวเลือก หากข้อมูลใน announce เคยระบุ tracker id ก็ควรจะกำหนดไว้ที่นี่

แทรคเกอร์จะตอบสนองด้วยเอกสาร “text/plain” ซึ่งมีพจนานุกรม bencode ที่มีค่าตั้งต้นดังรูป 2.4 ต่อไปนี้

Failure Reason	Warning Message	Interval	Min Interval
Tracker ID	Complete	Incomplete	Peers

รูปที่ 2.4 รูปแบบการตอบสนองของ Tracker

- failure reason: ถ้ามี จะอยู่ในรูปข้อความที่คนอ่านได้ (string)
- warning message: คล้ายกับ failure reason แต่กระบวนการจะดำเนินไปตามปกติ ข้อความจะถูกแสดงขึ้นเหมือน failure reason
- interval: ช่วงระยะห่างเป็นวินาทีที่เครื่องลูกข่ายควรรอก่อนจะส่งคำขอครั้งต่อไป
- min interval: ช่วงระยะอย่างต่ำในการประกาศ ถ้าระบุเครื่องลูกข่ายจะต้องไปประกาศในช่วงที่ต่ำกว่านี้
- tracker id: string ที่เครื่องลูกข่ายควรส่งมาในการประกาศครั้งต่อไป หากไม่มีและในการประกาศรอบก่อนได้ส่ง tracker id มา ไม่ต้องส่งค่าเก่า ให้ใช้ต่อไป
- complete: จำนวนของ peer ที่มีไฟล์โดยสมบูรณ์ หรือ seeder (จำนวนเต็ม)
- incomplete: จำนวนของ peer ที่มีไฟล์ไม่สมบูรณ์ หรือ lecher (จำนวนเต็ม)
- peers: มีค่าเป็นชุดข้อมูลของพจนานุกรม แต่ละชุดจะมีค่าตั้งต้นดังนี้
 - peer id: ค่าที่ตั้งเองของ peer ตามที่กล่าวไว้เบื้องต้น
 - ip: IP Address ของ peer (IPv4 หรือ IPv6) หรือ DNS name
 - port: Port number ของ Peer

ในปัจจุบันนอกจาก HTTP Tracker แล้ว ยังมีแบบที่เขียนขึ้นด้วย PHP และ C++ ซึ่งแตกต่างกันไปตามวิธีการทำงานของโค้ดและการพัฒนา เช่น XBNBT BNBT CBTT ที่เป็นตัวหลักๆในปัจจุบัน ส่วนแทรคเกอร์ที่เป็นโอเพนซอร์ส (Open Source) อื่นๆนั้น มาจากการการพัฒนาต่อจากแทรคเกอร์เหล่านี้

นอกจากนี้ ในโปรแกรมประยุกต์บางตัวยังรองรับ BitTorrent แบบ Trackerless ซึ่งใช้วิธีติดต่อกับ Peer อื่นๆที่ใช้โปรแกรมที่รองรับวิธีนี้เหมือนกัน โดยใช้การติดต่อค้นหา Distributed Hash Table ที่ตรงกันในขณะที่ทำงาน ซึ่ง peer เครื่องอื่นที่กำลังดำเนิน Torrent เดียวกันจะตอบรับ Query และสร้าง Peers Table ระหว่างกันโดยไม่ต้องมีแทรคเกอร์มาช่วย

2.1.4 การทำงานในรูปแบบ Peer-2-Peer ของเครื่องลูกข่าย

หลังจากมี Peers Table แล้ว โปรแกรมลูกข่าย(Client) จะติดต่อระหว่างกันโดยใช้ Peer Wire Protocol (TCP) เพื่อแลกเปลี่ยนชิ้นส่วนของข้อมูลที่ Torrent ที่ทำงานอยู่นั้นกล่าวถึง ซึ่งในจำนวน peer ที่มีทั้งหมดใน peer table นั้น โปรแกรมลูกข่ายจะไม่ได้ติดต่อไปทั้งหมดเพื่อรักษา Load Balance โดยมีการระบุสถานะกับแต่ละ peer ไว้ ซึ่งมี

- choked: บ่งบอกว่าเครื่องลูกข่ายนั้นถูกปิดกั้นโดย peer หรือไม่ ซึ่งในขณะที่ถูกปิดกั้น คำร้องขอต่างๆจะไม่ได้รับการตอบรับจนกว่าจะยุติการปิดกั้น เครื่องลูกข่ายควรระงับการร้องขอชิ้นส่วน และรู้ไว้ว่าคำร้องที่ไม่ได้รับการตอบจะถูกละเลยจาก peer
- interested: บ่งบอกว่า Peer สนใจที่จะขอข้อมูลจากเครื่องลูกข่าย ซึ่งทันทีที่ได้ยุติการปิดกั้น peer จะเริ่มส่งคำขอไปยังเครื่องลูกข่ายทันที

ซึ่งในความเป็นจริง ทั้ง Peer และเครื่องลูกข่ายนั้นนับเป็น peer เช่นกัน ดังนั้นข้อมูลสถานะจริงๆจะเป็นดังนี้

- am_choking: เครื่องลูกข่ายนี้กำลังปิดกั้น peer
- am_interested: เครื่องลูกข่ายนี้สนใจ peer
- peer_choking: เครื่อง peer กำลังปิดกั้นเครื่องลูกข่ายนี้
- peer_interested: เครื่อง peer สนใจเครื่องลูกข่าย

สถานะในการเริ่มการเชื่อมต่อจะเป็นการปิดกั้นและไม่สนใจจากทั้งสองฝ่าย ชิ้นส่วนจะถูกส่งมาสู่เครื่องลูกข่ายก็ต่อเมื่อ peer ไม่ได้ปิดกั้นเครื่องลูกข่าย และเครื่องลูกข่ายสนใจในตัว peer สิ่งสำคัญที่เครื่องลูกข่ายควรทำก็คือการบอก peer ทุก peer ถึงความสนใจที่มีต่อแต่ละ peer ซึ่งควรบอกไว้ทุกครั้งที่มีการเปลี่ยนแปลงแม้ว่า peer จะปิดกั้นอยู่ เพื่อให้ peer สามารถรับรู้ได้ว่าเครื่องลูกข่ายจะโหลดข้อมูลจาก peer ทันทีที่ยุติการปิดกั้น

ตารางที่ 2.1 สถานะการติดต่อกันของ Client กับ Peer

Peer to Client	Client to Peer			
Status	Interested	Not Interested	Choking	Unchoking
Interested	ต่างก็มีชิ้นที่อีกฝั่งยังขาด	มีชิ้นที่ peer ยังขาด	พิจารณาการ unchoke Peer	ส่งชิ้นส่วนให้ Peer
Not Interested	Peer มีชิ้นที่ยังขาด	หลังจากหนึ่งจะเลิกติดต่อ	ไม่จำเป็นต้อง unchoke	x
Choking	รอส่ง request	ไม่มีชิ้นส่วน	สถานะเริ่มต้น	x
Unchoking	รับชิ้นส่วนจาก Peer	x	x	x

สำหรับเลขจำนวนเต็มที่ส่งใน Peer Wire Protocol ถ้าไม่ได้ระบุไว้ จะอยู่ในรูป 4 byte big endian ซึ่งจะรวมถึงตัวระบุความยาวทุกตัวหลังจากการทำ handshake ใน Peer Wire Protocol จะมีการทำ handshake เพื่อเริ่มต้น หลังจากนั้น จะติดต่อกันผ่านข้อความที่ระบุความยาวไว้ ด้านหน้าตามรูปแบบที่กล่าวไป

การ Handshake เป็นข้อความที่จำเป็นต้องส่งเป็นข้อความแรก มีลักษณะดังรูปที่ 2.5 ข้างล่างนี้

PStrLen	PStr	Reserved	Info_Hash	Peer_ID
---------	------	----------	-----------	---------

รูปที่ 2.5 รูปแบบของ Handshake

- handshake: <pstrlen><pstr><reserved><info_hash><peer_id>
 - pstrlen: ความยาวของ pstr ในรูป single raw byte
 - pstr: String Identifier of the Protocol
 - reserved: 8 bytes ที่สงวนไว้ ในการใช้งานปัจจุบันให้ทุกไบต์เป็น 0

- info_hash: 20 bytes SHA1 hash ของส่วน info ใน metainfo file ซึ่งเป็น info_hash อันเดียวกับที่ใช้ส่งคำขอไปยังแทรคเกอร์
- peer_id: 20 bytes string ที่เป็นหมายเลขระบุตัวตนของเครื่อง อันเดียวกับที่ส่งไปบอกแทรคเกอร์
- ใน BitTorrent Protocol เวอร์ชัน 1.0 นั้น pstrlen=19 และ pstr="BitTorrent protocol" ผู้ที่เริ่มการเชื่อมต่อจะถูกคาดหวังให้ส่ง handshake ทันที ผู้รับอาจจะรอดู handshake ของผู้เริ่มหากสามารถรองรับการทำงาน torrent หลายงานได้พร้อมกัน ซึ่งผู้รับจะตอบสนองทันทีเมื่อได้ตรวจ info_hash ที่ส่งมา ถ้า info_hash ไม่ตรงกับ torrent ใดๆที่เครื่องลูกข่ายกำลังดำเนินอยู่ การเชื่อมต่อจะต้องยุติไป ถ้าหากผู้เริ่มได้รับการตอบที่มี peer_id ไม่ตรงกับที่คาดไว้ การเชื่อมต่อก็ยุติอีกเช่นกัน ทั้งนี้ peer_id ที่คาดนั้นนำมาจากที่ได้จากแทรคเกอร์

นอกจากนี้ยังมีส่วนของข้อความระหว่าง Peer เพื่อบอกสถานะระหว่างกัน รูปแบบจะเป็น <length prefix><message ID><payload> มีข้อความต่างๆ ดังรูปที่ 2.6 นี้

Length	Message ID	Payload
--------	------------	---------

รูปที่ 2.6 รูปแบบของข้อความที่ส่งระหว่าง Peer

- keep-alive: <len=0000>: เป็นข้อความที่ค่าเป็นศูนย์ ดังที่บอกไว้ในความยาวของข้อความ โดยไม่มีส่วนของ message ID และ payload ปกติ peer จะยุติการเชื่อมต่อหากไม่ได้รับข้อความใดๆเลยเป็นเวลาหนึ่ง ข้อความ keep-alive สามารถส่งได้เพื่อรักษาการเชื่อมต่อ โดยทั่วไป keep-alive message จะส่งทุกๆสองนาที
- choke: <len=0001><id=0>: ข้อความปิดกันมีความยาวจำกัดและไม่มี payload
- unchoke: <len=0001><id=1>: ข้อความยุติการปิดกันมีความยาวจำกัดและไม่มี payload
- interested: <len=0001><id=2>: ข้อความระบุสนใจมีความยาวจำกัดและไม่มี payload
- not interested: <len=0001><id=3>: ข้อความระบุไม่สนใจมีความยาวจำกัดและไม่มี payload
- have: <len=0005><id=4><piece index>: ข้อความ have มีความยาวจำกัด payload เป็นดัชนีฐานศูนย์ของชิ้นส่วนที่โหนดโดยสมบูรณ์และตรวจสอบค่า hash แล้ว
- bitfield: <len=0001+x><id=5><bitfield>: ข้อความ bitfield จะถูกส่งทันทีหลังจากการทำ handshake ก่อนข้อความอื่นๆเพื่อระบุชิ้นส่วนที่มีอยู่แล้วตามในดัชนี ซึ่งหากเครื่อง

ลูกข่ายไม่มีอะไรเลยก็สามารถละข้อความนี้ไว้ไม่ต้องส่งได้ ข้อความจะตั้งดัชนีที่ตรงกับชิ้นส่วนที่มีแล้วให้ค่าบิตเป็น 1 ส่วนที่ไม่มีจะเป็น 0 โดยนับจากบิตสูงในไบต์แรกลงไป บิตที่เหลือท้ายข้อความจะตั้งเป็น 0

- request: <len=0013><id=6><index><begin><length>: ข้อความร้องขอมีความยาวจำกัด ใช้ร้องขอ block ในส่วนของ payload จะมีข้อมูลดังต่อไปนี้
 - index: จำนวนเต็มระบุตำแหน่งชิ้นส่วนในดัชนีฐานศูนย์
 - begin: จำนวนเต็มระบุตำแหน่งไบต์ที่ถัดเข้าไปในชิ้นส่วน
 - length: จำนวนเต็มระบุความยาวที่ขอ ค่าปกติมักเป็น 2^{15} ไบต์ ค่าที่น้อยกว่าอาจนำมาใช้แต่มันจะไม่จำเป็นนอกจากในกรณีที่ชิ้นส่วนหารด้วย 2^{15} ไม่ลงตัว
- piece: <len=0009+x><id=7><index><begin><block>: ข้อความ piece มีความยาวหลากหลาย ซึ่ง x เป็นความยาวของ block ส่วน payload จะมีข้อมูลดังนี้
 - index: จำนวนเต็มระบุตำแหน่งชิ้นส่วนในดัชนีฐานศูนย์
 - begin: จำนวนเต็มระบุตำแหน่งไบต์ที่ถัดเข้าไปในชิ้นส่วน
 - block: block ของข้อมูล ซึ่งเป็นหน่วยย่อยของชิ้นส่วนที่ระบุไว้ในดัชนี
- cancel: <len=0013><id=8><index><begin><length>: ข้อความยกเลิกมีความยาวจำกัด ใช้เพื่อยกเลิกการร้องขอ block ค่าใน payload จะตรงกับข้อความ request ที่ใช้มักใช้ใน ช่วง End game algorithm
- port: <len=0003><id=9><listen-port>: ข้อความพอร์ตนี้ใช้ในรุ่นหลังๆ ของ Mainline ที่รองรับ DHT tracker (Trackerless) เพื่อระบุว่า peer รับฟัง DHT ที่พอร์ตไหน

2.2 พร็อกซีเซิร์ฟเวอร์

2.2.1 พร็อกซีเซิร์ฟเวอร์คืออะไร

พร็อกซีเซิร์ฟเวอร์หรืออาจเรียกกันว่าแคชเป็นอุปกรณ์เครือข่ายชนิดหนึ่ง ซึ่งจะทำให้การเก็บข้อมูลของหน้าเว็บเพจต่างๆ ที่เราเรียกมานั้นเอง โดยจะทำหน้าที่เป็นเสมือนที่เก็บข้อมูลหน้าเว็บเพจที่มีผู้เคยเรียกดูหน้านั้นไว้ในระบบของเซิร์ฟเวอร์และเมื่อมีผู้อื่นเรียกดูข้อมูลที่เป็นหน้าเว็บเพจตัวเดียวกันกับที่เคยมีเก็บไว้แล้วก็ไม่จำเป็นต้องไปเรียกอ่านข้อมูลมาจากเซิร์ฟเวอร์ตัวต้นฉบับจริงๆ ก็ได้ เพียงแค่ค้นหาข้อมูลเว็บเพจนั้นจากพร็อกซีก่อน ซึ่งถ้าหากหาพบหรือมีผู้อื่นเคยเรียกอ่านแล้วระบบพร็อกซีเซิร์ฟเวอร์ก็สามารถส่งข้อมูลนั้นมาให้เราได้ดูได้ทันที ซึ่งวิธีนี้จะทำให้เราสามารถเรียกดูข้อมูลได้เร็วกว่าการเรียก มาจากเว็บไซด์ต้นฉบับจริงๆ แต่ข้อเสียของการตั้งใช้ พร็อกซีเซิร์ฟเวอร์ ก็อาจจะมึบช้าเหมือนกัน คือ ข้อมูลที่มีการอัปเดตเร็วๆ

เช่นพวกเว็บบอร์ดต่าง ๆ อาจจะไม่มีการอัปเดตตามได้ทัน เพราะเราจะได้ข้อมูลที่มาจากร็อกซี่ ไม่ใช่มาจากต้นฉบับหรือแหล่งกำเนิดจริงๆ ในกรณีเช่นนี้ อาจจะทำการแก้ไขเบื้องต้น ได้โดยการกดปุ่ม Ctrl พร้อม ๆ กับการใช้เมาส์คลิกที่ปุ่ม Refresh จะเป็นการร้องขอ ข้อมูล หน้าเว็บเพจ นั้นๆ ใหม่จากเซิร์ฟเวอร์ต้นฉบับ

2.2.2 หลักการทำงานของพร็อกซี่เซิร์ฟเวอร์

เมื่อมีผู้ให้บริการทำการเรียกข้อมูลของเว็บไซต์โดยผ่านพร็อกซี่เซิร์ฟเวอร์ ในครั้งแรก พร็อกซี่เซิร์ฟเวอร์ จะทำการตรวจสอบว่า มีข้อมูลของเว็บไซต์นั้นมีอยู่หรือไม่ หากพบว่าไม่มีข้อมูล พร็อกซี่เซิร์ฟเวอร์ จะทำการเรียกข้อมูลนั้นจากเว็บไซต์แล้วเก็บไว้ในเครื่อง และเมื่อมีผู้ให้บริการทำการเรียกเว็บไซต์นี้อีกครั้ง พร็อกซี่เซิร์ฟเวอร์จะทำการส่งข้อมูลไปยังเครื่องของผู้ให้บริการทันที ในกรณีที่เว็บไซต์มีการอัปเดตข้อมูล พร็อกซี่เซิร์ฟเวอร์จะทำการตรวจสอบข้อมูลที่มีอยู่ว่าอัปเดตหรือไม่ และจะทำการอัปเดตข้อมูลใหม่ทันที ในกรณีที่มิผู้เรียกใช้บริการก็จะได้ข้อมูลที่อัปเดตอยู่เสมอ

2.2.3 การจัดการแคชของพร็อกซี่เซิร์ฟเวอร์

ในการจัดการแคชของ พร็อกซี่เซิร์ฟเวอร์ จะมีอยู่ 2 แบบ คือ

2.2.3.1 Passive Caching

จะเป็นการจัดการแคชแบบทั่วไป คือจะบันทึกข้อมูลลงในแคชของพร็อกซี่ เฉพาะเมื่อไคลเอนต์มีการขอใช้ข้อมูลผ่านทางพร็อกซี่เซิร์ฟเวอร์เท่านั้น

2.2.3.2 Active Caching

จะใช้เทคนิคที่สูงกว่า โดย พร็อกซี่เซิร์ฟเวอร์ จะพิจารณาเองว่าควรจะเก็บข้อมูลจากเว็บเซิร์ฟเวอร์ใด และเก็บไว้ในแคชเมื่อใด โดยไม่จำเป็นต้องรอให้ไคลเอนต์ขอใช้ข้อมูลเข้ามาก่อน

2.2.4 โพรโทคอลที่ใช้ในการจัดการแคชของพร็อกซี่

ส่วนโปรโตคอลที่ใช้ในการจัดการแคชของพร็อกซีนั้นมีอยู่ 2 แบบด้วยกันคือ

2.2.4.1 Internet Cache Protocol (ICP : RFC 2186)

Internet Cache Protocol หรือ ICP ได้รับการพัฒนามาจากมหาวิทยาลัย Southern California ซึ่งปัจจุบัน ICP ได้ถูกพัฒนาขึ้นเพื่อใช้งานร่วมกับ IPv6 ได้ด้วย (แต่ยังไม่ได้รับการรับรองเป็นมาตรฐาน) การใช้งาน ICP นี้จะใช้โปรโตคอลพื้นฐานร่วมกัน 3 ชุด โดยแยกหมายเลขพอร์ตให้ต่างกัน คือหมายเลขพอร์ตแรกจะใช้กับโปรโตคอล

HTTP เพื่อให้ไคลเอนต์ติดต่อเข้ามายัง พร็อกซีเซิร์ฟเวอร์ และหมายเลขพอร์ตที่สองจะ ใช้กับโปรโตคอล UDP เพื่อสื่อสารแบบ ICP ระหว่าง พร็อกซีเซิร์ฟเวอร์ ด้วยกัน ส่วน หมายเลขพอร์ตสุดท้ายจะใช้กับโปรโตคอล TCP เพื่ออ่านข้อมูลเว็บเพจจากเลขของ พร็อกซีเซิร์ฟเวอร์ อย่างไรก็ตาม ICP ก็ยังมีจุดอ่อนอยู่ตรงที่ไม่สามารถรองรับการขยายตัวของระบบได้ดีพอ เนื่องจากข้อมูลที่แลกเปลี่ยนกันระหว่าง พร็อกซีเซิร์ฟเวอร์ จะเพิ่มขึ้นอย่างมากและซ้ำซ้อนกัน เมื่อมีการเพิ่มจำนวน พร็อกซีเซิร์ฟเวอร์ มากขึ้น ซึ่งเมื่อมีการร้องขอใช้ข้อมูลจาก พร็อกซีเซิร์ฟเวอร์ นั้น ถ้าหากค้นหาไม่พบในเครื่องแรก ก็จะต้อง ค้นหาในเครื่องที่สองและต่อไปเรื่อยๆ ทำให้ประสิทธิภาพการทำงานลดลง

2.2.4.2 Cache Array Routing Protocol (CARP : RFC 3040)

Cache Array Routing Protocol หรือ CARP ได้พัฒนาให้มีความสามารถมากขึ้น โดยมุ่งที่จะลดภาระในการติดต่อสื่อสารระหว่าง พร็อกซีเซิร์ฟเวอร์ ให้น้อยลง โดยการ ให้พร็อกซีหลายๆชุดจัดเรียงตัวกัน โดยมีตารางเก็บรายละเอียดและหมายเลขไอพีของ พร็อกซีเซิร์ฟเวอร์ทั้งหมดไว้ ส่วนในการค้นหาข้อมูลนั้น ไคลเอนต์ก็จะส่ง URL ที่ ต้องการใช้งานผ่านเข้ามายัง พร็อกซีเซิร์ฟเวอร์ ซึ่งจะใช้ฟังก์ชันการ Hash ที่ใช้หา ตำแหน่งของข้อมูลที่ต้องการ นำมาใช้หา URL ดังกล่าวว่าอยู่ในเลขของ พร็อกซีเซิร์ฟเวอร์ใด แต่ถ้าหากไม่พบก็จะไปดึงมาจากเว็บเซิร์ฟเวอร์โดยตรง ซึ่ง CARP นี้ สามารถเรียกได้อีกอย่างว่า Queryless Distributed Caching และในปัจจุบันผลิตภัณฑ์ทั้ง ของบริษัทไมโครซอฟท์และเน็ตสเคปก็ได้สนับสนุนโปรโตคอลแบบ CARP ทั้งสิ้น

2.2.5 ประโยชน์ของการใช้พร็อกซีเซิร์ฟเวอร์

ผู้ให้บริการสามารถเรียกดูข้อมูลจากเว็บไซต์ต่าง ๆ ได้รวดเร็วขึ้น และช่วยประหยัดเวลา ในการใช้งานอินเทอร์เน็ต ทั้งนี้ก็เพราะพร็อกซีเซิร์ฟเวอร์ ก็จะสามารถใช้ข้อมูลที่เก็บไว้จากการ ร้องขอของผู้ใช้รายแรกมาส่งให้แก่ผู้ใช้รายอื่นๆ ได้เลยโดยไม่จำเป็นต้องทำการร้องขอไปยังเว็บ เซิร์ฟเวอร์อีกครั้ง ทำให้สามารถประหยัดได้ทั้งเวลาและแบนวิดของเครือข่าย นอกจากนี้ ยังได้ ข้อมูลที่มีความถูกต้องและทันสมัยอยู่ตลอดเวลา

นอกจากนี้พร็อกซียังเกี่ยวข้องกับเรื่องของความปลอดภัยของผู้ใช้ด้วย ตามปกติแล้วถ้า ผู้ใช้งานที่มีความรู้พื้นฐานเกี่ยวกับอินเทอร์เน็ตบ้าง ก็คงจะเคยได้ยินหรือรู้จักคำว่าไอพีแอดเดรส มาบ้าง คอมพิวเตอร์ที่อยู่บนเน็ตเวิร์ก (ที่ใช้โปรโตคอล TCP/IP) หรือบนอินเทอร์เน็ตจะมีไอพี แอดเดรสเอาไว้เพื่อเป็นการบอกที่อยู่ของเครื่องคอมพิวเตอร์แต่ละเครื่องบนเน็ตเวิร์กนั้นๆ หรือ บนอินเทอร์เน็ต ในบางครั้งเวลาที่เราเปิดเว็บเบราว์เซอร์แล้วไปยังเว็บไซต์ต่าง ๆ นั้น พวก เว็บไซต์เหล่านี้จะเก็บข้อมูลหลายๆอย่างของผู้ที่เข้ามาเยี่ยมชมเว็บไซต์ โดยส่วนมากเก็บ log เป็น

ไฟล์ไว้ด้วยความสามารถของตัวเว็บเซิร์ฟเวอร์เองที่มีอยู่แล้วหรือ Script ต่างๆ ข้อมูลหลายๆอย่าง ที่พูดมานั้น หนึ่งในนั้นก็จะมีไอพีแอดเดรสรวมอยู่ด้วย ในบางครั้งตัวพร็อกซีเซิร์ฟเวอร์สามารถช่วย ปิดบัง ข้อมูลต่างๆรวมทั้งไอพีแอดเดรสของเรา เช่น การเรียกดูเว็บเพจจะเป็นพร็อกซีเซิร์ฟเวอร์ เองที่ไปเรียกดูจากเว็บไซต์นั้นๆ แทนที่จะเป็นเครื่องคอมพิวเตอร์ของเราที่ไปเรียกดูโดยตรง เว็บไซต์เหล่านั้นจึงได้แต่ข้อมูลของพร็อกซีเซิร์ฟเวอร์ไปแทนที่จะได้จากเครื่องคอมพิวเตอร์ของ ผู้ใช้งานที่เรียกดู

อีกประการหนึ่งพร็อกซียังมีคุณสมบัติในการจำกัดสิทธิที่จะเข้าถึงเว็บไซต์บางแห่ง ที่มีเนื้อหาไม่สมควรเข้าชม การจำกัดยูสเซอร์ใช้งานในเวลาที่น่าอึดใจจากเวลางาน หรือ ข้อจำกัดอื่น ๆ ที่ทำให้สิ้นเปลืองค่าใช้จ่ายไปโดยไม่ก่อให้เกิดประโยชน์ คุณสมบัติเช่นนี้ล้าพัง อินเทอร์เน็ตเคยยังไม่เพียงพอ จึงจะต้องอาศัยพร็อกซีเซิร์ฟเวอร์ (พร็อกซีเซิร์ฟเวอร์) เข้ามา ช่วยเสริมอีกแรงหนึ่ง ซึ่งภายในลินุกซ์ทุกดิสทริบิวชันจะมีโปรแกรมพร็อกซีเซิร์ฟเวอร์ที่มี ประสิทธิภาพสูงให้มาพร้อมแล้ว คือ โปรแกรม Squid

โปรแกรม Squid เป็น พร็อกซีเซิร์ฟเวอร์ ที่มีคุณสมบัติในการจำกัด ควบคุมการแอกเซส เข้าสู่เว็บไซต์ภายนอกองค์กรได้เป็นอย่างดีและมีประสิทธิภาพ ที่เรียกว่า Access Control List (ACL) ซึ่งเป็นการนิยามชื่อลิสต์ขึ้นแทนคุณสมบัติของสิ่งที่ต้องการอ้างอิง จากนั้นจึงตั้ง ข้อกำหนดลงไปว่าต้องการให้ลิสต์นั้นสามารถแอกเซสผ่านพร็อกซีได้หรือไม่ ดังนั้นการที่เสริม การทำงานของอินเทอร์เน็ตเซิร์ฟเวอร์ด้วย Squid พร็อกซีเซิร์ฟเวอร์ จึงเป็นการควบคุมการเข้าสู่ อินเทอร์เน็ตของผู้ใช้งานในองค์กรได้ตามต้องการ และยังช่วยเพิ่มประสิทธิภาพให้แก่ระบบอีกด้วยเพราะ Squid จะมีคุณสมบัติเป็น HTTP Object cache ที่ช่วยเก็บข้อมูลจากเว็บไซต์ภายนอก ไว้ในหน่วยความจำ (RAM และฮาร์ดดิสก์) ของตัวเซิร์ฟเวอร์เองอีกด้วย ช่วยให้การเรียก เว็บไซต์ที่เคยเข้าถึงมาก่อนทำได้รวดเร็วยิ่งขึ้น เนื่องจากมีข้อมูลบางส่วนของเว็บเพจที่ ยังคงอยู่ในแคชนั่นเอง

2.2.6 Transparent Proxy คืออะไร

พร็อกซีเซิร์ฟเวอร์นั้นผู้ใช้งานทั่วไปจำเป็นต้องเข้าไปเซตค่าพร็อกซีเซิร์ฟเวอร์ ที่ตัว บราวเซอร์ก่อน ซึ่งอาจจะมีผู้ใช้งานที่ไม่ได้ทำเช่นนั้น ส่วน Transparent Proxy นั้นผู้ใช้งาน ไม่จำเป็นต้องเซตค่าอะไรในเครื่องคอมพิวเตอร์ของตัวเอง ตัวของ Transparent Proxy นั้นจะใช้ หลักของ iptables ในการ redirects ผู้ที่ติดต่อผ่านมาทางพอร์ต http ให้ไปยัง พร็อกซีเซิร์ฟเวอร์ ได้เองโดยอัตโนมัติ

2.3 โปรแกรม Squid

บนระบบปฏิบัติการลินุกซ์ มีโปรแกรมพร็อกซีเซิร์ฟเวอร์ที่มีประสิทธิภาพสูงมาก คือ โปรแกรม Squid ซึ่งเป็นโปรแกรมประเภท Proxy Caching Server สำหรับการให้บริการ Web Caching Service คือ จะคอยรับคำร้องขอบริการจากเครื่องลูกข่าย และส่งผ่านไปยังเซิร์ฟเวอร์ปลายทางที่เหมาะสม ข้อมูลต่าง ๆ ที่ผ่านเข้ามาจะถูกสำเนาเก็บไว้ในหน่วยความจำแคช และディスク ดังนั้นเมื่อมีการร้องขอข้อมูลซ้ำอีกในครั้งต่อมาก็จะสามารถนำข้อมูลในแคชมาให้บริการได้รวดเร็วกว่าการติดต่อไปยังเซิร์ฟเวอร์โดยตรง ช่วยให้การใช้ช่องทางสื่อสารข้อมูลลงได้

2.3.1 ความสามารถของโปรแกรม Squid

- Proxying และ caching ของ HTTP, FTP และ URLs อื่นๆ
- Proxying สำหรับ SSL
- Cache hierarchies
- ICP, HTCP, Cache Digests
- Transparent caching
- WCCP (squid v2.3 และเวอร์ชันที่ใหม่กว่านั้น)
- Extensive access controls
- HTTP server acceleration
- SNMP
- Caching of DNS lookups

2.3.2 Access Log ของโปรแกรม squid

เมื่อเครื่องลูกข่ายเรียกใช้งานเว็บไซต์ภายนอก จะทำการติดต่อผ่านพอร์ต 8080 มายังเครื่องเซิร์ฟเวอร์ลินุกซ์ที่รันโปรแกรม Squid ไว้ เราสามารถมอนิเตอร์ดูความเคลื่อนไหวของการเรียกใช้งานดังกล่าวได้จาก LOG ไฟล์ของ squid ด้วยคำสั่ง

```
tail -f /var/log/squid/access.log (แล้วแต่จะกำหนดตำแหน่งของ LOG ไฟล์)
```

จะเห็นข้อความแสดงเหตุการณ์ต่าง ๆ ที่เกิดขึ้นเกี่ยวกับโปรแกรม Squid ได้ดังรูป ซึ่งเราสามารถตีความหมายได้ เช่น

TCP_MISS หมายถึง การร้องขอนั้นไม่มีการแคชข้อมูลไว้ กรณีนี้จะต้องไป GET มาจากเว็บไซต์ปลายทางมาจริง

TCP_HIT หรือ TCP_MEM_HIT หมายถึง มีข้อมูลที่เครื่องลูกข่ายร้องขอมาอยู่แล้วในแคช กรณีนี้จะเอาข้อมูลที่อยู่ในแคชส่งกลับไปให้ผู้ร้องขอ

```
1025162259.349 6 192.168.0.254 TCP_MEM_HIT/200 440 GET
http://www.thairath.co.th/picuse/rmore_it.gif - NONE/- image/gif
1025162259.454 11 192.168.0.254 TCP_MEM_HIT/200 653 GET
http://www.thairath.co.th/picuse/foot_it.gif - NONE/- image/gif
1025162259.758 3 192.168.0.254 TCP_HIT/200 9413 GET
http://www.thairath.co.th/link/schedule.jpg - NONE/- image/jpeg
1025162259.970 17 192.168.0.254 TCP_HIT/200 8550 GET
http://www.thairath.co.th/link/IT.banner.gif - NONE/- image/gif
1025162263.053 3321 192.168.0.254 TCP_MISS/200 400 GET
http://adserve.inet.co.th/jnserver/SITE=thairath.co.th/AREA=thairath.index/A
AMSZ=195x39 - DIRECT/203.150.14.102 application/x-javascript
1025162263.165 2992 192.168.0.254 TCP_MISS/200 430 GET
http://www.thairath.co.th/pichead/dotrd.gif - DIRECT/203.151.217.25
image/gif
```

ไฟล์ access.log ของโปรแกรม Squid นี้จะมีขนาดเพิ่มขึ้นเรื่อย ๆ จนอาจทำให้เนื้อที่ดิสก์เต็ม และสร้างปัญหาให้กับระบบได้ ดังนั้นจึงควรใช้กำหนดให้ Squid ทำการเปลี่ยนไฟล์ Log ด้วยคำสั่ง

```
squid -k rotate
```

โดยอาจจะเขียนเป็นเชลล์สคริปต์เพื่อให้ Log ไฟล์เกิดการหมุนเวียนกันไป ในแต่ละสัปดาห์ก็ได้ แต่ถ้าไม่จำเป็นต้องใช้ข้อมูลจาก Log ก็สามารถยกเลิกการเก็บ Log ไปได้โดยใช้สัวิร์ด ดังนี้

```
cache_access_log /dev/null
```

นอกจาก access.log แล้ว ไฟล์ที่สำคัญอีกไฟล์หนึ่งก็คือ cache.log มีหน้าที่เก็บสถานะและข้อมูลเกี่ยวกับการดีบั๊ก โปรแกรม Squid ซึ่งมีประโยชน์อย่างมากในการแก้ไขปัญหาต่าง ๆ ที่เกิดขึ้น

2.3.3 Cache Dir แคลชของโปรแกรม Squid

พื้นที่เก็บข้อมูลแคชบนดิสก์ของ Squid จะอยู่ที่ /var/spool/squid (อาจไม่อยู่ในตำแหน่งนี้เสมอไป ขึ้นอยู่กับการกำหนดของผู้ติดตั้งโปรแกรม) พื้นที่นี้ควรแยกออกจากพาร์ติชันของระบบอย่างเด็ดขาด เนื่องจากพื้นที่นี้จะมีข้อมูลอ่าน เขียนอยู่ตลอดเวลาทำให้มีโอกาสเสื่อมสภาพมากกว่าส่วนอื่นๆ หากเกิดข้อผิดพลาดขึ้นแล้วจะได้ไม่สร้างความเสียหายไปยังส่วนอื่น ๆ ที่สำคัญต่อระบบด้วย นอกจากนี้หากพิจารณาเรื่องของความเร็ว และประสิทธิภาพแล้ว ควรสร้างพื้นที่นี้ในฮาร์ดดิสก์ที่มีความเร็วสูง และหากเป็นไปได้ควรใช้ฮาร์ดดิสก์ชนิด SCSI จะเหมาะสมกว่าชนิด IDE

หากเราต้องการตรวจดูว่า พื้นที่ที่เราใช้งานเป็นดิสก์แชนมีการใช้เนื้อที่ไปแล้วมากน้อยแค่ไหน สามารถตรวจสอบได้จากคำสั่ง

```
du -sh /var/spool/squid
```

หรือกรณีที่แบ่งแยกพาร์ติชันนี้ออกมา ดังที่กล่าวไว้ข้างต้น สามารถตรวจสอบได้ด้วยคำสั่ง df -h อีกคำสั่งหนึ่ง

2.3.4 โปรแกรมยูทิลิตี้ของโปรแกรม Squid

โปรแกรม cachemgr.cgi เป็นยูทิลิตี้ของ Squid ที่ช่วยในด้านการรายงานสถานะ ค่าสถิติต่าง ๆ เกี่ยวกับคอนฟิกและประสิทธิภาพของโปรแกรม Squid โดยจะมีติดตั้งมาพร้อมกันกับ Squid แล้ว โปรแกรมนี้ถูกออกแบบให้ใช้งานผ่านเว็บ (Web Interface)

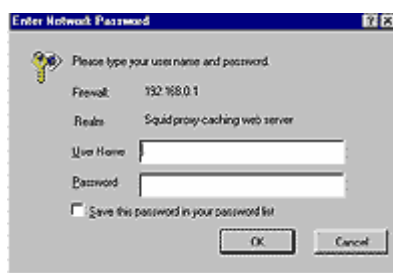
โปรแกรมนี้จะอยู่ที่ไดเรกทอรี /usr/lib/squid และเราจะต้องสำเนาไปไว้ที่พื้นที่ CGI ของเว็บเซิร์ฟเวอร์ คือ /var/www/cgi-bin เพื่อให้สามารถเรียกใช้งานได้

```
cp /usr/lib/squid/cachemgr.cgi /var/www/cgi-bin
```

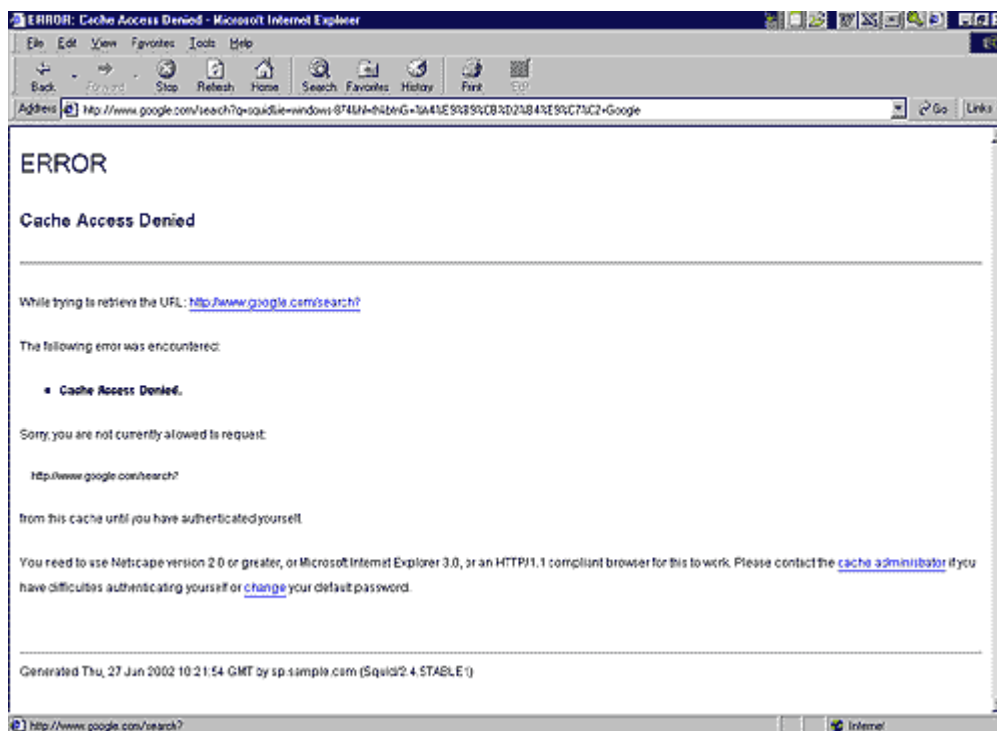
เมื่อได้ติดตั้งโปรแกรมนี้ไว้ที่พื้นที่ CGI ของเว็บเซิร์ฟเวอร์แล้ว ให้เปิดโปรแกรม Web Browser ไปที่แอดเดรส <http://192.168.0.1/cgi-bin/cachemgr.cgi> โดยที่ 192.168.0.1 เป็นไอพีแอดเดรสของเครื่องเว็บเซิร์ฟเวอร์และ Squid ของเรา จากนั้นจะต้องป้อนชื่อของ cache manger และรหัสผ่านตามที่ได้อำหนดเอาไว้ในไฟล์ squid.conf เมื่อคลิกที่ปุ่ม Continue แล้วเห็นรายละเอียดต่าง ๆ เป็นข้อ ๆ ดังรูป ซึ่งเป็นข้อมูลสถิติทั้งหมดเกี่ยวกับ Squid Proxy Server ของเรา ดังรูปที่ 2.7 ถึง 2.9

2.3.5 User Authentication ตรวจสอบสิทธิ์ของผู้ใช้งานเว็บ

คุณสมบัติที่น่าสนใจอีกประการหนึ่งของ Squid ก็คือการตรวจสอบสิทธิ์ของผู้ใช้งานด้วยชื่อ และรหัสผ่าน ซึ่งจะช่วยให้ผู้จัดการระบบสามารถควบคุมการใช้งานเว็บของผู้ใช้แต่ละคนได้ เมื่อนำมารวมเข้ากับคุณสมบัติด้าน Firewall หรือ Access Control List แล้ว จะสามารถกำหนดเงื่อนไขได้หลากหลายมาก เช่น กำหนดให้นายสมชายสามารถเข้าได้เฉพาะเว็บไซต์ที่เกี่ยวข้องกับงานของบริษัทเท่านั้น นางสาวสมศรีสามารถเข้าเว็บได้เฉพาะเวลา 12.00 น. - 13.00 น. เท่านั้น เป็นต้น ซึ่งการตรวจสอบรายชื่อของผู้ใช้จะต้องอาศัยโปรแกรมภายนอกเข้ามาเสริมการทำงาน โดยที่ใน Squid เองก็มีโปรแกรมจำพวกนี้ให้มาบ้างแล้วจำนวนหนึ่ง ซึ่งจะอยู่ในไดเรกทอรี /usr/lib/squid ได้แก่ smb_auth.sh, pam_auth เป็นต้น ซึ่งแต่ละโปรแกรมก็จะมีเทคนิคการตรวจสอบรายชื่อผู้ใช้ที่แตกต่างกันออกไป บ้างก็ใช้ระบบรายชื่อจาก Windows NT บ้างก็ใช้ระบบรายชื่อจากตัว Linux เอง ซึ่งผู้จัดการระบบสามารถเลือกใช้ได้ตามความต้องการดังรูปที่ 2.10 และรูปที่ 2.11



รูปที่ 2.10 แสดงกรอบโต้ตอบของโปรแกรม mySquid_Auth



รูปที่ 2.11 แสดงข้อความของ Squid เมื่อผู้ใช้เข้าสู่เว็บไซต์ที่ไม่ได้รับอนุญาต

อย่างไรก็ตามเมื่อได้นำโปรแกรมเหล่านี้มาใช้งานจึงได้พบข้อบกพร่อง และข้อจำกัดที่ไม่ตรงตามความต้องการใช้งาน ผู้เขียนจึงต้องเขียนโปรแกรมขึ้นเอง ชื่อโปรแกรม mySquid_Auth ซึ่งมีระบบจัดเก็บรายชื่อของผู้ใช้เป็นของตนเองโดยอิสระ และสามารถจัดเก็บสถิติการใช้งานของผู้ใช้เพื่อตรวจสอบได้ในภายหลัง โปรแกรมนี้สามารถดาวน์โหลดได้จากเว็บไซต์ <http://www.itdestination.com>

Squid เป็นโปรแกรมพร็อกซีเซิร์ฟเวอร์ที่มีประสิทธิภาพสูง มีคุณสมบัติที่น่าสนใจมากมาย สามารถช่วยให้การใช้แบนวิธของเครือข่ายเป็นไปอย่างคุ้มค่าเรื่องราวของ Squid ยังมีสิ่งที่ต้องศึกษาอีกมากมาย ไม่ว่าจะเป็นเรื่องของการจัดสรรโควตาเพื่อจำกัดการใช้แบนวิธ การใช้งาน Squid ในโหมด HTTPd Accelerator การทำงานในสภาพแวดล้อมแบบ Multi-Level Web Caching การทำงานในแบบ Transparent Proxy และเรื่องราวอื่นๆ อีกมากมาย โดยสามารถศึกษาเพิ่มเติมได้จากเว็บไซต์ของโปรเจกต์ Squid ที่ <http://www.squid-cache.org>

2.4 โปรแกรม SquidGuard

โปรแกรม SquidGuard คือโปรแกรมที่เป็น plugin สำหรับโปรแกรม Squid ซึ่งมีความสามารถในการทำ filter, redirect และการทำ access controller โดยโปรแกรม SquidGuard นั้น เป็นโปรแกรมฟรีแวร์ที่ทำงานบนระบบปฏิบัติการ UNIX

2.4.1 โปรแกรม SquidGuard นำมาใช้ในด้าน

- จำกัดการเข้าเว็บไซต์สำหรับผู้ใช้งานโดยพิจารณาจากไอพีแอดเดรส และ/หรือ URLs
- บล็อกการเข้าเว็บไซต์ที่อยู่ใน list หรือ blacklist
- บล็อกการเข้า URLs ที่ตรงกับ list ของ regular expression หรือ words
- Redirect จากเว็บไซต์ที่บล็อกไปยังหน้าเว็บไซต์ที่ต้องการได้

2.4.2 ความสามารถของโปรแกรม SquidGuard

- define different time spaces ได้แก่
 - time of day (00:00-08:00 17:00-24:00)
 - day of the week (sa)
 - date (1999-05-13)
 - date range (1999-04-01-1999-04-05)
 - date wildcards (*-01-01 *-05-17 *-12-25)
- group source (users/clients) ได้แก่
 - IP address range with
 - Prefix notation (172.16.0.0/12)
 - Netmask notation (172.16.0.0/255.240.0.0)
 - First-last notation (172.16.0.11-172.16.0.35)
 - Address lists (172.16.134.54 172.16.156.23 ...)
 - Domain lists (foo.bar.com ...) *)
 - User id lists (weho sdgh dfhj asef ...) **)
 - Positively (within business-hours)
 - Negatively (outside leisure-time)
- group destinations (URLs/servers) ได้แก่
 - Domains รวมทั้ง subdomains (foo.bar.com)
 - Hosts (host.foo.bar.com)
 - Directory URLs รวมทั้ง subdirectories (foo.bar.com/some/dir)
 - File URLs (foo.bar.com/somewhere/file.html)
 - Regular expression ((expr1|expr2|...))
 - Positively (within business-hours)
 - Negatively (without leisure-time)

- rewrite/redirect URLs ได้แก่
 - string/regular expression editing โดย
 - silent squid redirecting rewrite (s@from@to@[i])
 - visible client redirecting rewrite (s@from@to@[i]r ***)
 - URL replacement โดย
 - Silent squid redirect to a common URL(redirect “new_url”)
 - Visible client redirect to a common URL (redirect “302:new_url”
***)
- define access control lists (acl)
 - giving each source (user/client) group
 - a pass list
 - acceptable destination groups (good-dests ...)
 - unacceptable destination groups (!bad-dests ...)
 - block IP address URLs (enforce the use of domain names)
(!in-addr)
 - wildcards/nothing (any|all|none)
 - optionally a common rewrite rule set สำหรับ source group
 - optionally a default replacement URL สำหรับบล็อก destination
สำหรับ source group
 - link the acl to a given time space
 - positively (within business-hours)
 - negatively (outside leisure-time)
 - defining a fallback/default ruleset
- มีตัวเลือกสำหรับ logging ดังนี้
 - Source/client group declaration เป็น log สำหรับทุกๆ translation ของ group
(log “file”)
 - Destination group declaration. เป็น log สำหรับที่ match กับ blacklist (log
“file”)
 - Rewrite rule group declaration เป็น log สำหรับทุกๆ translation ของ rule set

2.5 เว็บเซอร์วิส

2.5.1 ที่มาของเว็บเซอร์วิส

ปัจจุบันท่ามกลางสถานะเศรษฐกิจที่เปลี่ยนแปลงประกอบกับการแข่งขันทางธุรกิจที่สูงขึ้น ปัจจัยหลายๆ อย่างเข้ามามีอิทธิพลต่อรูปแบบการดำเนินธุรกิจ ทว่าแนวโน้มที่เห็นได้ชัดเจนสำหรับธุรกิจในยุคใหม่ คือ การทำธุรกิจในลักษณะอิเล็กทรอนิกส์หรือที่เรียกว่า พาณิชยอิเล็กทรอนิกส์ (E-Business) ซึ่งเป็นวิธีการทำธุรกิจที่น่าสนใจและกำลังแพร่หลายในหลายๆ ธุรกิจ รวมทั้งยังเป็นปัจจัยหนึ่งที่มีส่วนทำให้องค์กรทางธุรกิจนั้นๆ ยืนหยัดอยู่ได้ องค์กรทางธุรกิจต่างๆ ได้เรียนรู้ที่จะนำข้อมูลและทรัพยากรด้านเทคโนโลยีสารสนเทศ มาใช้เป็นเครื่องมือในการพัฒนาธุรกิจของตนมากยิ่งขึ้น เพื่อเป็นฐานที่มั่นคงและการเติบโตทางเศรษฐกิจในอนาคต

องค์กรทางธุรกิจต่างๆ เริ่มมองเห็นวิธีที่ดีที่สุดสำหรับการพัฒนาประสิทธิภาพของการปฏิบัติงาน ลดค่าใช้จ่าย และสานสัมพันธ์ที่มีกับลูกค้ารวมถึงซัพพลายเออร์รายสำคัญให้แน่นแฟ้นยิ่งขึ้น โดยสร้างวิธีการดำเนินธุรกิจแบบใหม่ที่ใช้เทคโนโลยีผสมผสานกับเครือข่ายอินเทอร์เน็ตที่แพร่หลายในปัจจุบัน ซึ่งเทคโนโลยีที่กำลังได้รับความสนใจและยังเป็นเครื่องมือที่สนับสนุนความต้องการดังกล่าวในธุรกิจยุคใหม่ก็คือ เว็บเซอร์วิส

เป็นที่ยอมรับกันว่าเทคโนโลยีสารสนเทศและเครือข่ายอินเทอร์เน็ตเป็นเครื่องมือธุรกิจที่จำเป็นในการดำเนินธุรกิจยุคใหม่ไปแล้ว ซึ่งเว็บแอปพลิเคชันเป็นพระเอกหรือเป็นเครื่องมือสำคัญในการทำธุรกิจบนโลกของอินเทอร์เน็ต แต่ปัจจุบันเว็บแอปพลิเคชันกำลังจะกลายเป็นเพียงตัวประกอบ ด้วยการที่องค์กรธุรกิจเริ่มหันมาสนใจเทคโนโลยีใหม่อย่างเว็บเซอร์วิส ซึ่งเข้ามาช่วยเพิ่มประสิทธิภาพให้กับระบบการดำเนินธุรกิจขององค์กร และยังสามารถทำงานร่วมกับเว็บแอปพลิเคชันเดิมได้เป็นอย่างดี

เพื่อให้องค์กรธุรกิจต่างๆ ได้เห็นแนวทางในการนำเว็บเซอร์วิสไปใช้ในธุรกิจของตน แต่ละองค์กรจำเป็นต้องเข้าใจความหมายของเว็บแอปพลิเคชันและเว็บเซอร์วิส เพื่อที่จะได้เห็นความแตกต่างและนำไปประยุกต์ใช้งานได้อย่างถูกต้องและมีประสิทธิภาพมากที่สุด

2.5.2 เว็บแอปพลิเคชันคืออะไร

เว็บแอปพลิเคชัน (Web application) คือ โปรแกรมที่อยู่ในเว็บเซิร์ฟเวอร์ที่คอยให้บริการสิ่งที่ร้องขอ (request) จากทางไคลเอนต์ผ่านโปรโตคอล HTTP ซึ่งจะแสดงผลที่ร้องขอในรูปแบบของ HTML page ผ่านทางบราวเซอร์ ซึ่งก็คือเว็บไซต์ต่างๆ ที่เราใช้บริการอยู่นั่นเอง

เว็บแอปพลิเคชันสามารถตอบสนองความคิด Distributed Processing ได้ในระดับหนึ่ง ซึ่งก็คือ การแบ่งการประมวลผลไว้ที่ฝั่งไคลเอนต์และฝั่งเซิร์ฟเวอร์ และมักจะมีการใช้ฐานข้อมูล (database) ควบคู่กับการทำเว็บแอปพลิเคชันไปด้วยตามความต้องการในการทำ E-Business และ

E-Commerce ที่กำลังเป็นที่นิยมในปัจจุบัน แต่ปัญหาที่ตามมาคือ เรื่องของการจ่ายเงินหรือที่เรียกว่า E-payment หรือ Payment-Gateway ซึ่งเว็บแอปพลิเคชันที่ทำ E-Commerce ต้องใช้บริการจากธนาคารออนไลน์ ในการจัดเก็บเงินกับลูกค้าด้วยเทคโนโลยีนี้ การใช้บริการเก็บเงินจากธนาคารออนไลน์ จำเป็นที่ผู้ค้าจะต้องไปทำการตกลงกับธนาคารและเขียนโปรแกรมให้ตรงตามมาตรฐานที่ธนาคารออนไลน์กำหนดไว้

2.5.3 เว็บเซอร์วิสคืออะไร

เว็บเซอร์วิส คือ แอปพลิเคชันหรือโปรแกรมที่ทำงานอย่างใดอย่างหนึ่ง ในลักษณะให้บริการ โดยจะถูกเรียกใช้งานจากแอปพลิเคชันอื่นๆ ในรูปแบบ RPC(Remote Procedure Call) ซึ่งการให้บริการจะมีเอกสารที่อธิบายคุณสมบัติของบริการกำกับไว้ โดยภาษาที่ถูกใช้เป็นตัวในการแลกเปลี่ยนคือ XML ทำให้เราสามารถเรียกใช้ส่วนประกอบใดๆ ก็ได้ ในระบบหรือ platform ใดๆ ก็ได้ บนโปรโตคอล HTTP ซึ่งเป็นโปรโตคอลสำหรับอินเทอร์เน็ต อันเป็นช่องทางที่ได้รับการยอมรับทั่วโลกในการติดต่อสื่อสารกันระหว่างแอปพลิเคชันกับแอปพลิเคชันในปัจจุบัน

การทำงานของเว็บเซอร์วิส ประกอบไปด้วย มาตรฐานหลัก 4 อย่าง ซึ่งสามารถอธิบายอย่างง่าย ๆ ได้ดังนี้

1. XML (Extensible Markup Language) เป็นภาษามาตรฐานที่ทุกระบบสนับสนุน ทำให้ข้อมูลที่มีโครงสร้างของภาษา XML จะถูกนำไปประมวลผลต่ออย่างอัตโนมัติได้อย่างง่ายดาย ภาษา XML จึงถูกนำมาใช้เป็นภาษามาตรฐานในการแลกเปลี่ยนข้อมูลของเว็บเซอร์วิส
2. SOAP (Simple Object Access Protocol) หรือ โซพ เป็นมาตรฐานของเทคโนโลยี Distributed Objects แบบหนึ่ง โดยทำหน้าที่ส่งข้อมูลผ่านอินเทอร์เน็ต ในรูปแบบของ XML ทำให้เรียกใช้งานโปรแกรมข้ามระบบผ่านทางอินเทอร์เน็ตได้
3. WSDL (Web Services Description Language) เป็นภาษามาตรฐานที่ใช้สำหรับอธิบายการใช้งานโปรแกรมที่เปิดให้บริการ ซึ่งเขียนขึ้นตามแบบมาตรฐาน XML ดังนั้น WSDL จึงเป็นเสมือนคู่มือให้กับระบบ เพื่อเรียนรู้วิธีการเรียกใช้งานเว็บเซอร์วิส
4. UDDI (Universal Description, Discovery, and Integration) เป็นระบบมาตรฐานในการอธิบายและค้นหาเว็บเซอร์วิส โดยเป็นตัวกลางให้ Provider มาลงทะเบียนไว้ โดยใช้ไฟล์ WSDL บอกรายละเอียดของบริษัทและบริการที่มีให้ ทำให้ Requestor สามารถค้นหาและทราบว่าบริษัทมีผลิตภัณฑ์และบริการอะไรบ้าง สามารถติดต่อขอดำเนินธุรกิจการค้ากับบริษัทได้โดยอัตโนมัติผ่านทางเว็บเซอร์วิส

จากมาตรฐานทั้ง 4 อย่างที่กล่าวข้างต้นสามารถสรุปลำดับขั้นตอนการทำงานของเว็บเซอร์วิส ได้ดังนี้

1. Provider จัดทำระบบหรือบริการที่เป็นเว็บเซอร์วิสขึ้นมา
2. ทำการลงทะเบียนเว็บเซอร์วิสกับหน่วยงานที่ให้บริการระบบ UDDI
3. นำ WSDL ไฟล์ไปไว้ในระบบ UDDI ที่ได้ลงทะเบียนไว้
4. Requestor ทำการค้นหาระบบหรือบริการที่ต้องการจากระบบ UDDI
5. เมื่อ Requestor ได้พบระบบหรือบริการที่ต้องการจะนำไฟล์ WSDL ไปเรียนรู้วิธีการเรียกใช้ผ่านระบบของตน
6. Requestor ทำการติดต่อและเรียกใช้ระบบหรือบริการจาก Provider ได้โดยตรงผ่าน SOAP ในระบบของตน

ถ้าพิจารณาลักษณะการทำงานและความสามารถของเว็บเซอร์วิสแล้ว เราสามารถสรุปได้ว่าเว็บเซอร์วิสก็คือ วัฒนาการอีกก้าวหนึ่งของเว็บแอปพลิเคชันนั่นเอง

2.5.4 เปรียบเทียบระหว่างเว็บแอปพลิเคชันและเว็บเซอร์วิส

เมื่อเราเข้าใจความหมายและการทำงานเว็บแอปพลิเคชันและเว็บเซอร์วิสแล้ว จะเห็นว่าเครื่องมือทั้งสองต่างใช้ HTTP Protocol หรืออินเทอร์เน็ตเป็นช่องทางในการสื่อสารเหมือนกัน แต่มีวัตถุประสงค์ต่างกัน โดยเว็บแอปพลิเคชันใช้เพื่อการแลกเปลี่ยนไฟล์ HTML ระหว่าง เว็บเซิร์ฟเวอร์ แต่เว็บเซอร์วิสเป็นการแลกเปลี่ยน “บริการ” ระหว่างระบบสารสนเทศ ผ่านเว็บเซิร์ฟเวอร์

ในเรื่องของความสามารถโดยส่วนใหญ่จะใช้เว็บแอปพลิเคชัน ในการติดต่อกับผู้ใช้ผ่านทางเว็บเบราว์เซอร์ เพื่อนำเสนอข้อมูลและการทำงานธุรกรรมต่างๆ ส่วนเว็บเซอร์วิสจะทำหน้าที่ในการติดต่อกับเว็บเซิร์ฟเวอร์ เพื่อแลกเปลี่ยนข้อมูลและการทำงานหรือใช้บริการข้ามระบบกันโดยใช้ Web Application หรือ Application Interface ในการติดต่อกับผู้ใช้ นอกจากนี้เว็บเซอร์วิสยังสามารถทำงานกับระบบต่างๆ ได้มากกว่า 1 ระบบ ในขณะที่เว็บแอปพลิเคชันไม่สามารถทำได้โดยตรง ซึ่งสามารถสรุปการเปรียบเทียบได้ดังตารางที่ 2.2

ตารางที่ 2.2 สรุปการเปรียบเทียบระหว่างเว็บเซอร์วิสกับเว็บแอปพลิเคชัน

หัวข้อ	Web Services	Web Applications
การเชื่อมต่อ	program-program	human-program
ภาษาที่ใช้	XML	HTML
รายชื่อการให้บริการ	ค้นหาผ่าน UDDI	ค้นหาผ่าน search engine
ขอบเขตการใช้งาน	Business-to-Business (B2B)	Business-to-Customer (B2C)
โปรโตคอล(Protocol)	SOAP+HTTP	HTTP

หลังจากที่ได้ทำความเข้าใจความหมายและข้อเปรียบเทียบระหว่างเว็บแอปพลิเคชันและเว็บเซอร์วิสแล้ว ความแตกต่างระหว่างเครื่องมือทั้งสองจะเห็นได้ชัดในเรื่องของความสามารถ แต่ในการนำไปใช้งานจำเป็นจะต้องประยุกต์ใช้เครื่องมือทั้งสองชนิดร่วมกัน เพื่อให้เกิดประสิทธิภาพมากที่สุด